

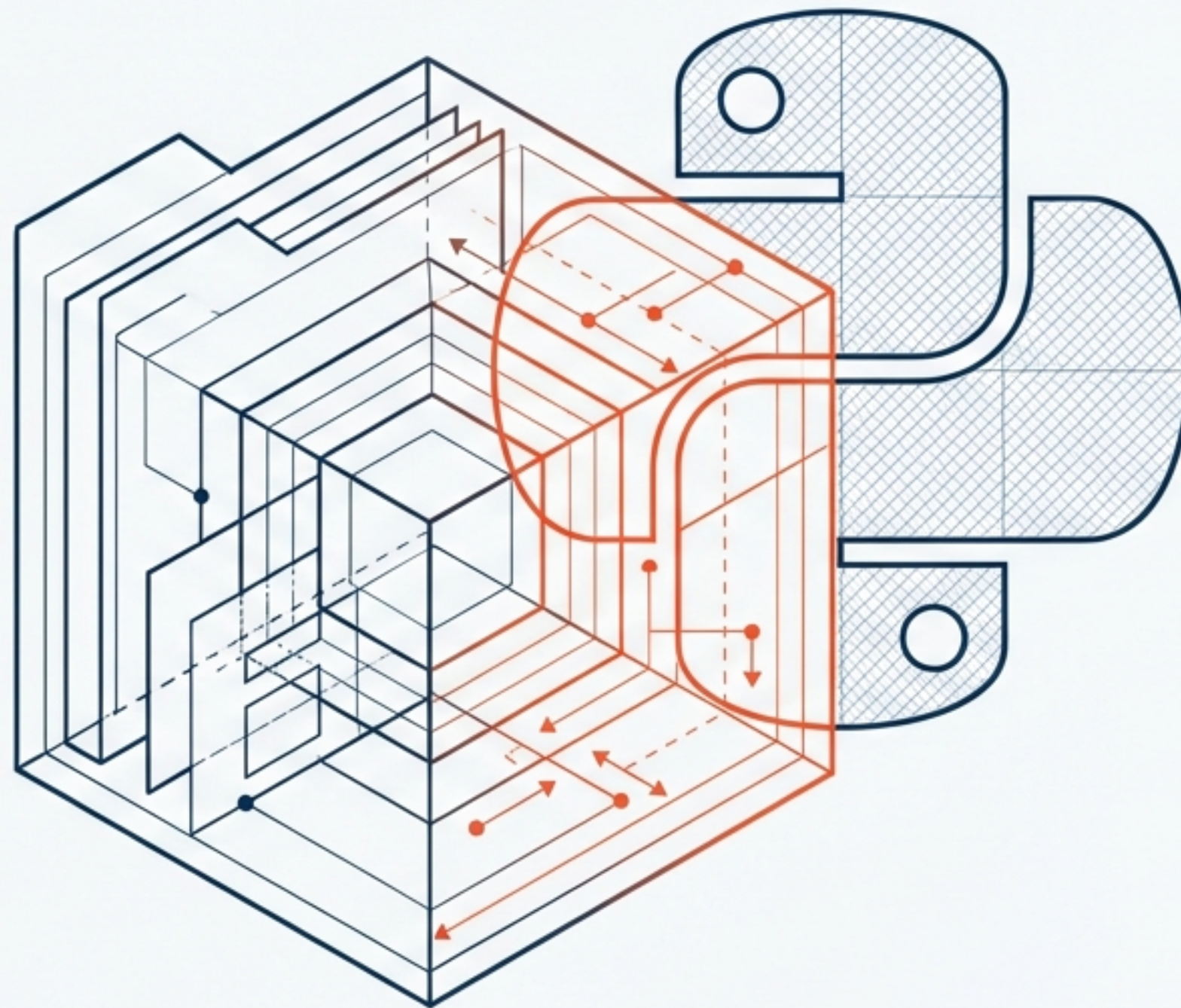
PROJECT BRIEF | v0.2.26

Git Repository Reader Service

A Pure-Python, Zero-Dependency Inspection Engine.

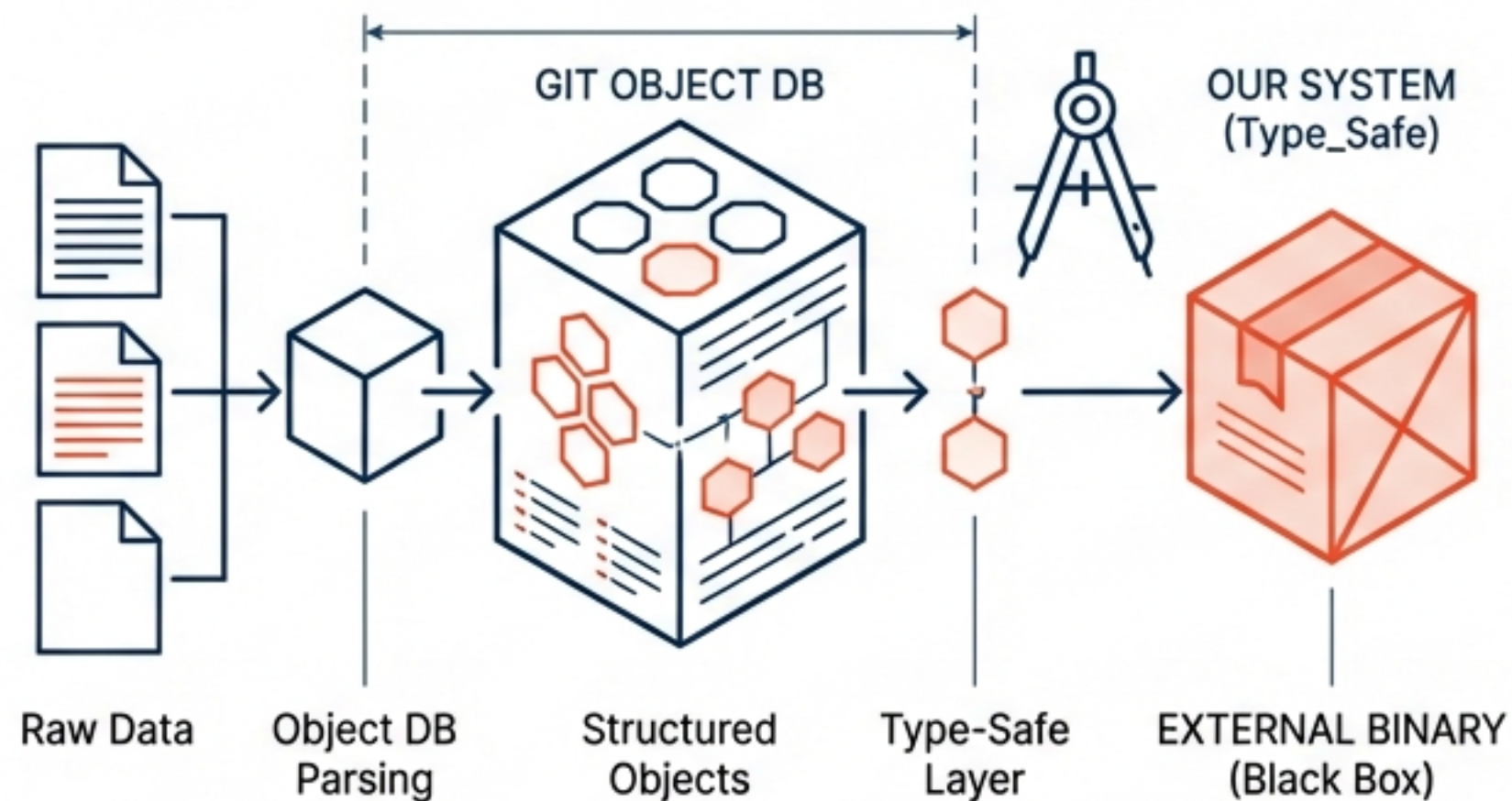
Foundation: `osbot_utils.type_safe`

Status: Initial Design



THE CORE MISSION

We are building a system to parse **Git's internal object database** directly. The goal is to **inspect repository state**, maintain full **type safety**, and eliminate the "**black box**" of external binaries.



CAPABILITIES

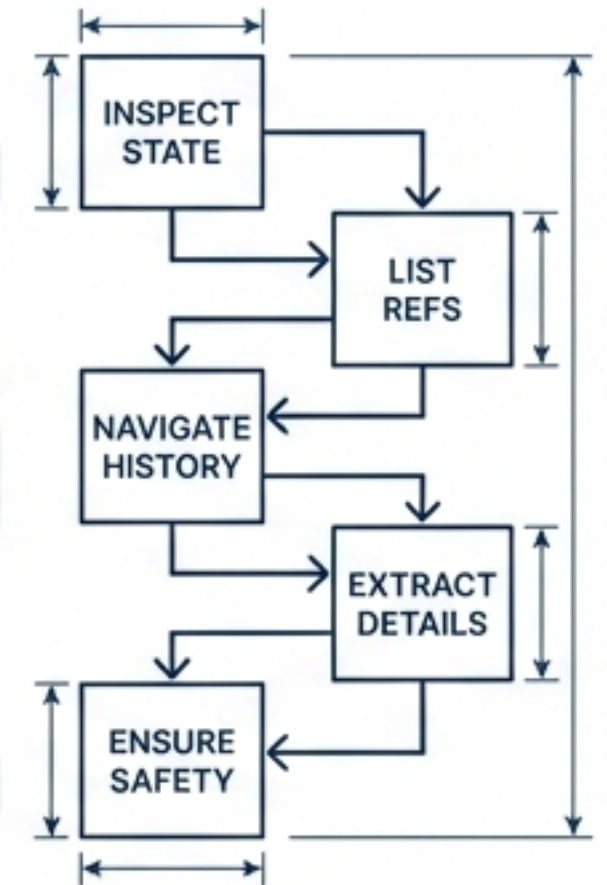
- ✓ **1. Inspect**
Repository state without the git binary.
[No git CLI dependency]
- ✓ **2. List**
All branches and their current commit SHAs.

```
> _list_branches()  
[SHA: e3b8c44298...]
```
- ✓ **3. Navigate**
Walk commit history within any branch.

```
> _walk_history(branch="main")
```
- ✓ **4. Extract**
Commit details (message, author, timestamp) and changed files.

```
CommitInfo { message: String, author: Signature, ... }  
> _get_diff()
```
- ✓ **5. Maintain**
Full type safety via the Type_Safe architecture (no raw primitives).

```
enum GitObject { Commit(CommitData), Tree(TreeData), ... } [No Raw Primitives]
```

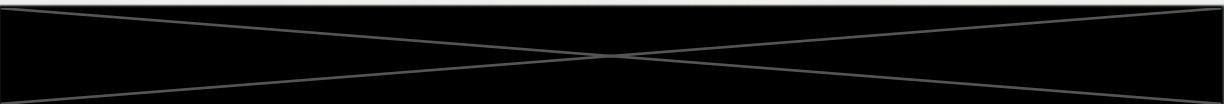


Target: Zero external dependencies—no git CLI, no GitPython, no dulwich.

Comparative Advantage: Why Direct .git Reading?

The Old Way: External Dependencies



- Requires 'git' CLI installed
- Subprocess overhead per operation
- Opaque error messages
- 
- **Security Risk:** Subprocess injection vulnerabilities

The New Way: Direct Reading



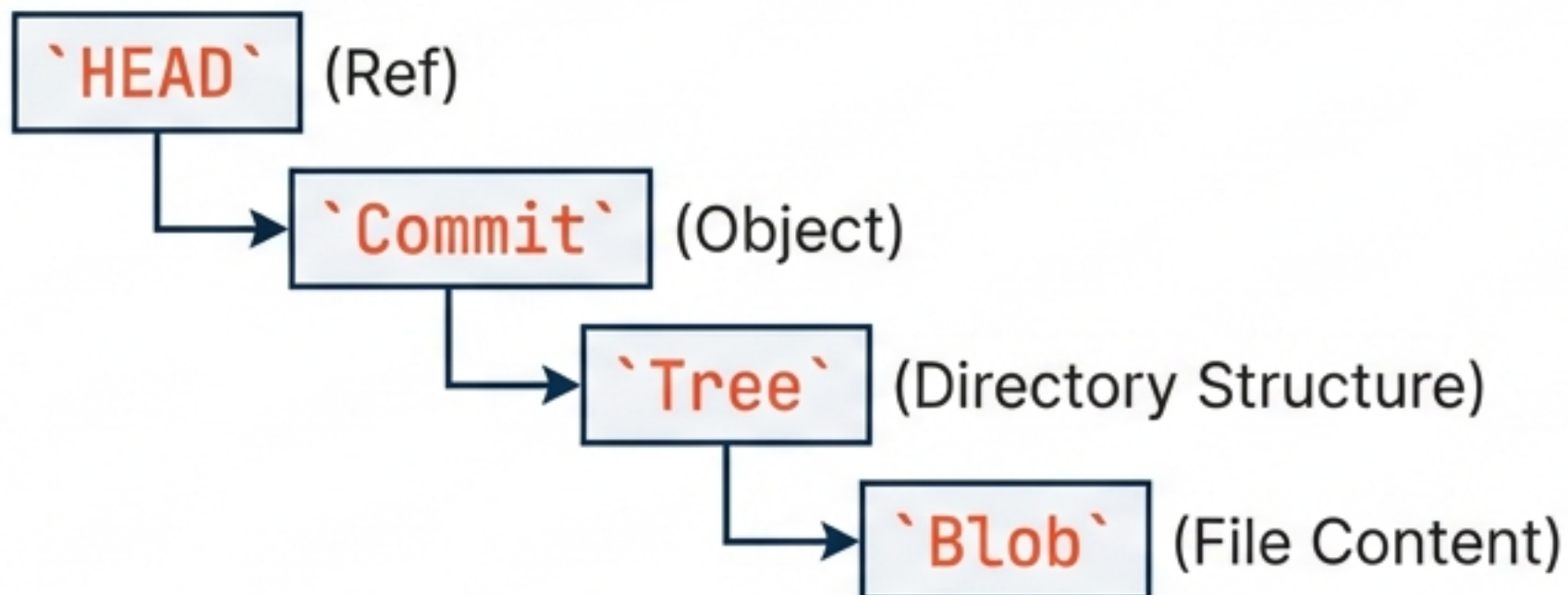
- **Zero system dependencies**
- **Native Python speed** (No Inter-Process Communication)
- Self-contained, minimal footprint
- **Control:** Full control over parsing and error handling

The Key Insight: Git is a Database

Git's object database is simple—zlib-compressed files with a well-documented format.



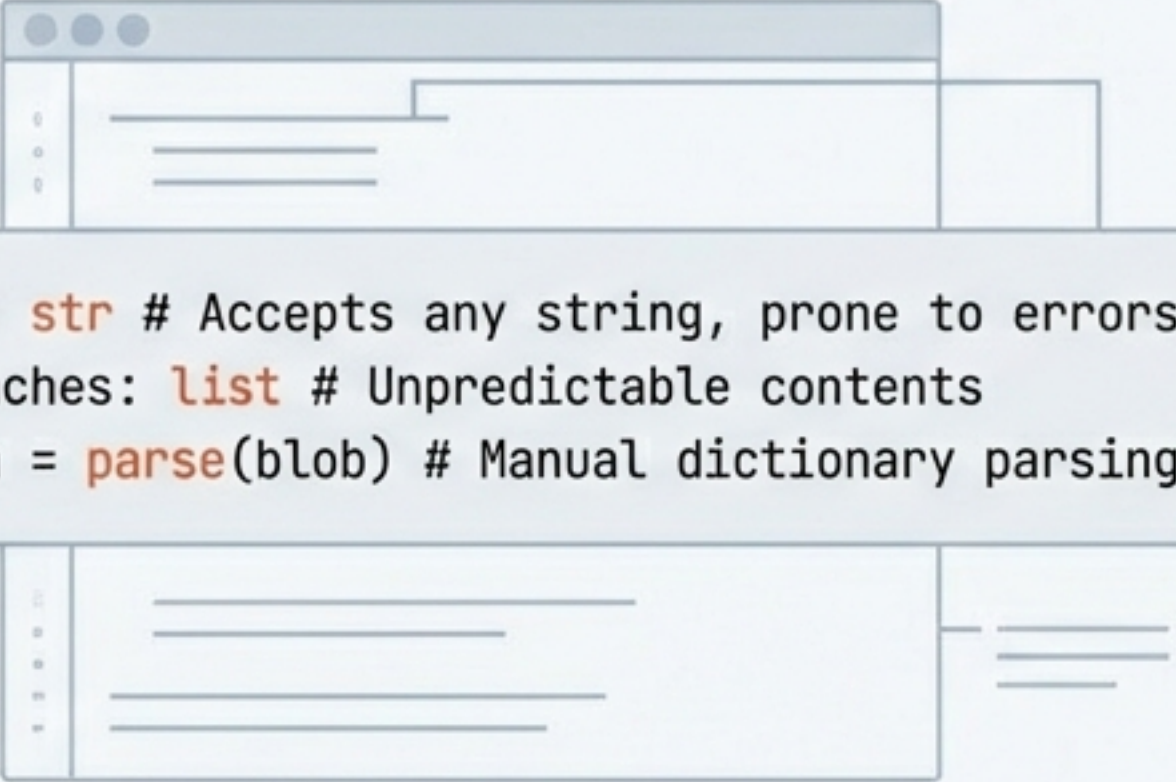
The Git Object Hierarchy



The Type_Safe Philosophy

Raw Python strings are dangerous. We enforce structure at the atomic level to prevent silent corruption.

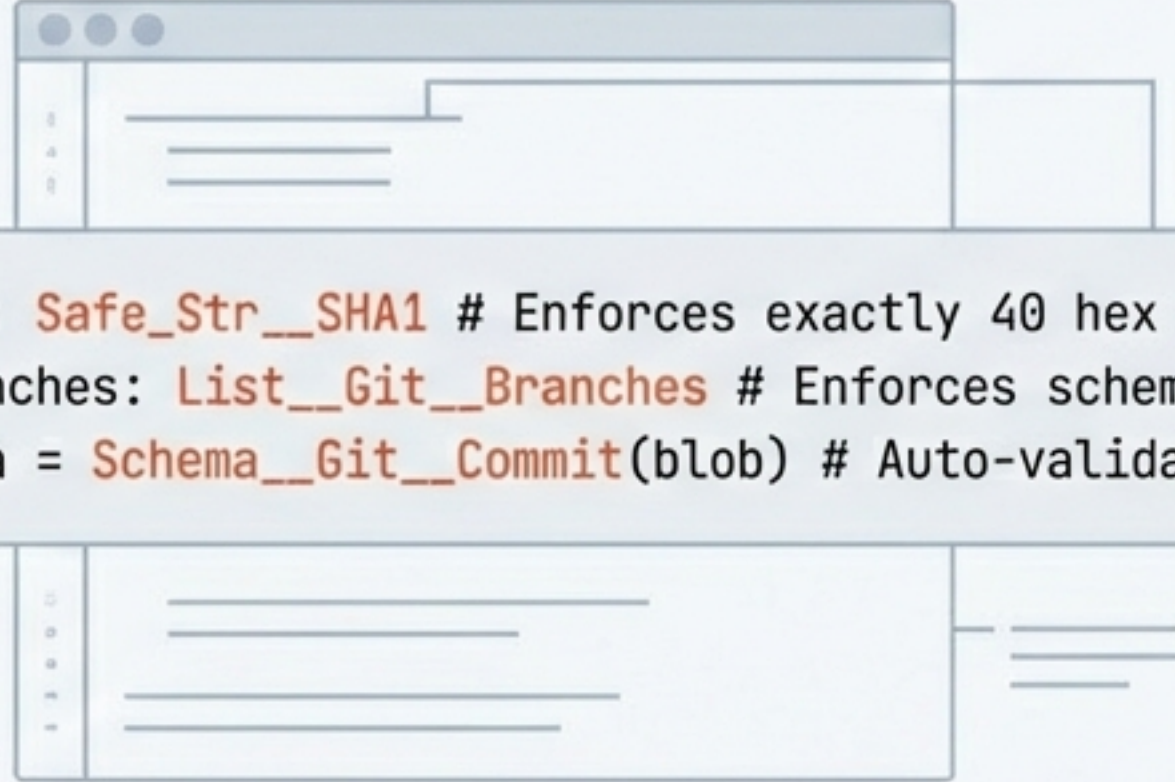
Raw Python Approach



```
sha: str # Accepts any string, prone to errors  
branches: list # Unpredictable contents  
data = parse(blob) # Manual dictionary parsing
```

Risk: Silent data corruption.

Type_Safe Approach



```
sha: Safe_Str__SHA1 # Enforces exactly 40 hex chars  
branches: List__Git__Branches # Enforces schema  
data = Schema__Git__Commit(blob) # Auto-validation
```

Benefit: Runtime validation & Auto-serialization.

Primitive Analysis: Reuse vs Create

Reuse (From osbot_utils)

Primitive	Validation Rule
Safe_Str__SHA1	^[a-fA-F0-9]{40}\$
Safe_Str__Git__Branch	git-check-ref-format rules
Safe_Str__SHA1__Short	Exactly 7 chars (Display only)

Create (New Requirements)

Primitive	Constraints
Safe_Str__Git__Commit_Message	Max 64KB, preserve formatting
Safe_Str__Git__Tree_Path	Max 4096 chars
Safe_Str__Git__Author_Line	Name <email> ts tz format

Schema Design: The Commit Object

Schema_Git_Commit

"tree_sha"	"Safe_Str__SHA1"
"parent_shas"	"List[Safe_Str__SHA1]"
"author"	"Safe_Str__Git__Author_Line"
"message"	"Safe_Str__Git__Commit_Message"
"sha"	"Safe_Str__SHA1"
"sha_short"	"Safe_Str__SHA1__Short"

Design Decision

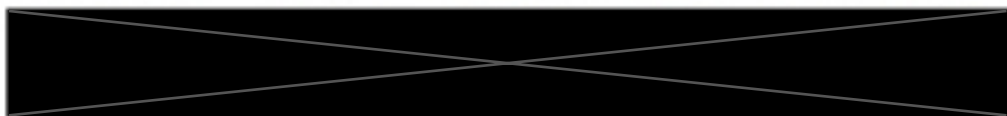
The schema includes both 'sha' (40-char) and 'sha_short' (7-char). The service populates 'sha_short' automatically during parsing to avoid repetitive slicing in UI layers.

The Logic Flow: Reading History



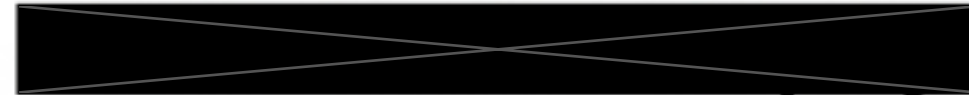
MVP Scope Definition

IN SCOPE (The Brief)



- List Branches (Return local branches with tip SHAs)
- List Commits (Walk history with depth limits)
- Read Details (Message, author, timestamp, parents)
- File Diffs (Identify Added/Modified/Deleted files)

OUT OF SCOPE (Future)



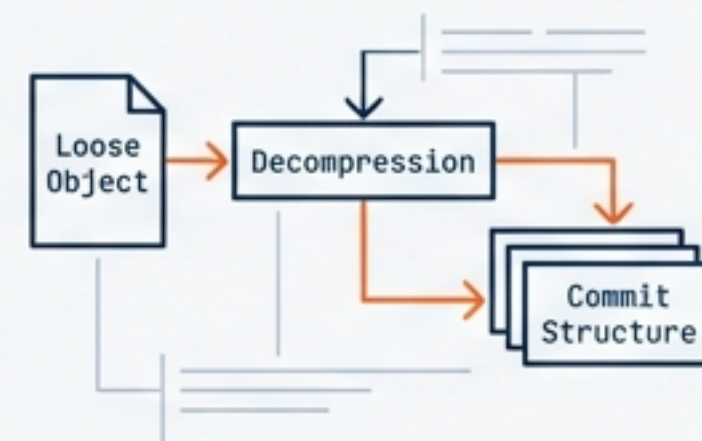
- Remote branches (refs/remotes/*)
- Tags (refs/tags/*)
- Blob content retrieval (Reading file contents)
- Writing to the repository
- Full Packfile delta reconstruction (Partial only)

Roadmap: Foundations

Milestones 1 & 2 - Priority P0

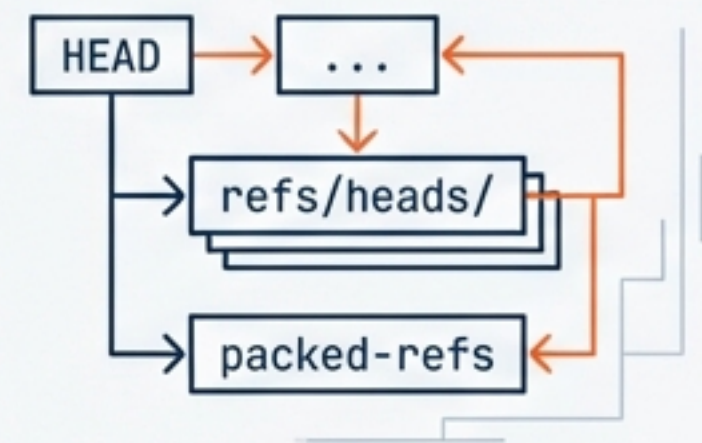
Milestone 1: Core Object Reading

- Deliverable: `Git_Object_Reader` (loose objects)
- Deliverable: `parse_commit` logic
- Challenge: Handling zlib edge cases and malformed objects



Milestone 2: Reference Reading

- Deliverable: `Git_Ref_Reader` (HEAD resolution)
- Deliverable: `branch_names()`
- Challenge: Handling detached HEAD states and packed-refs format



20% Completion



Roadmap: The Tree & Diffs

Milestone 3 - Priority P0

Focus: Tree Parsing & File Changes

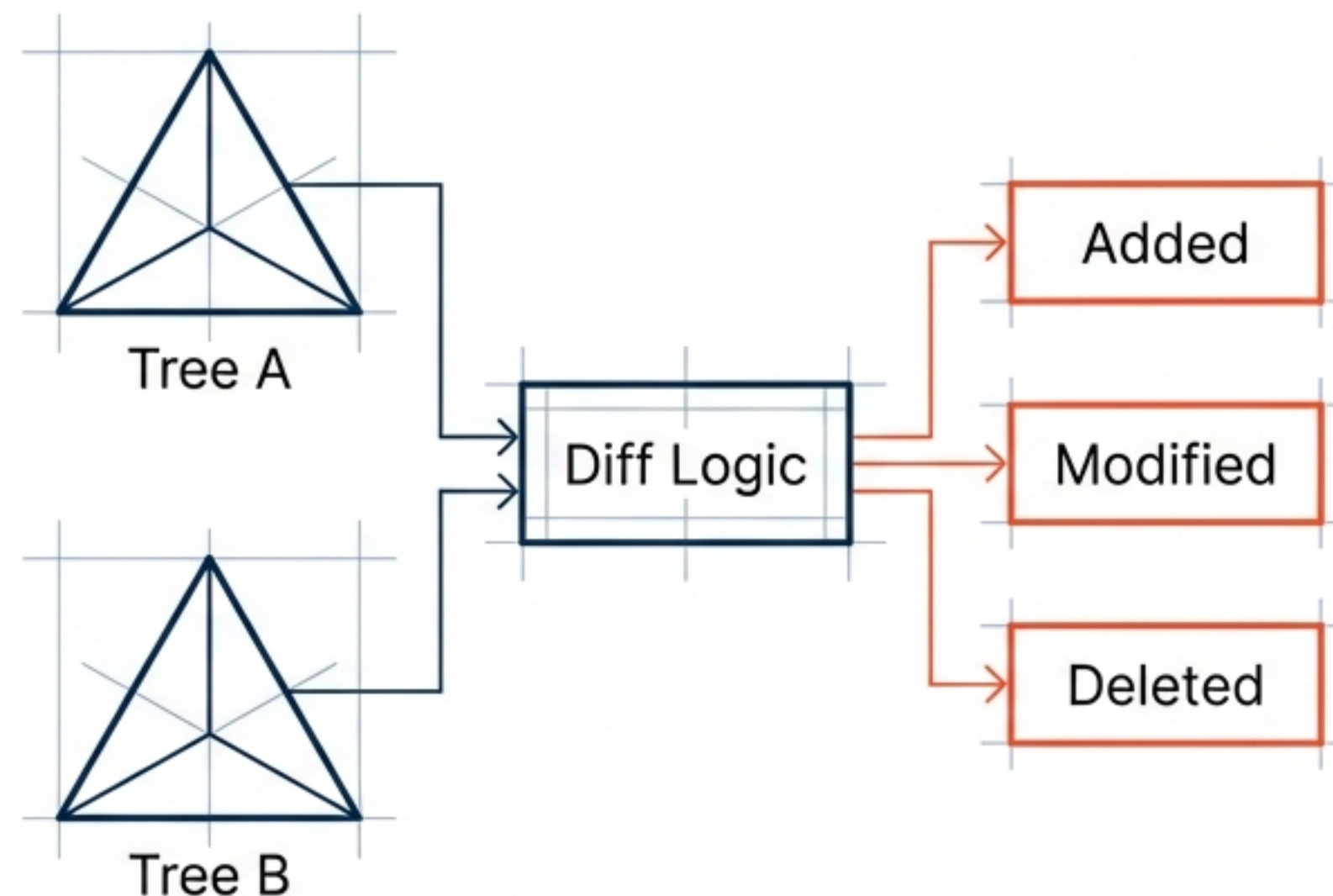
- Deliverables:

- `Git_Object__Parser.parse_tree()`

`Git_Object__Parser.parse_tree()`

- Recursive tree flattening

`Git_Repo__Reader__Service.files_changed()`



Key Challenge: Binary tree format parsing and recursive flattening logic.

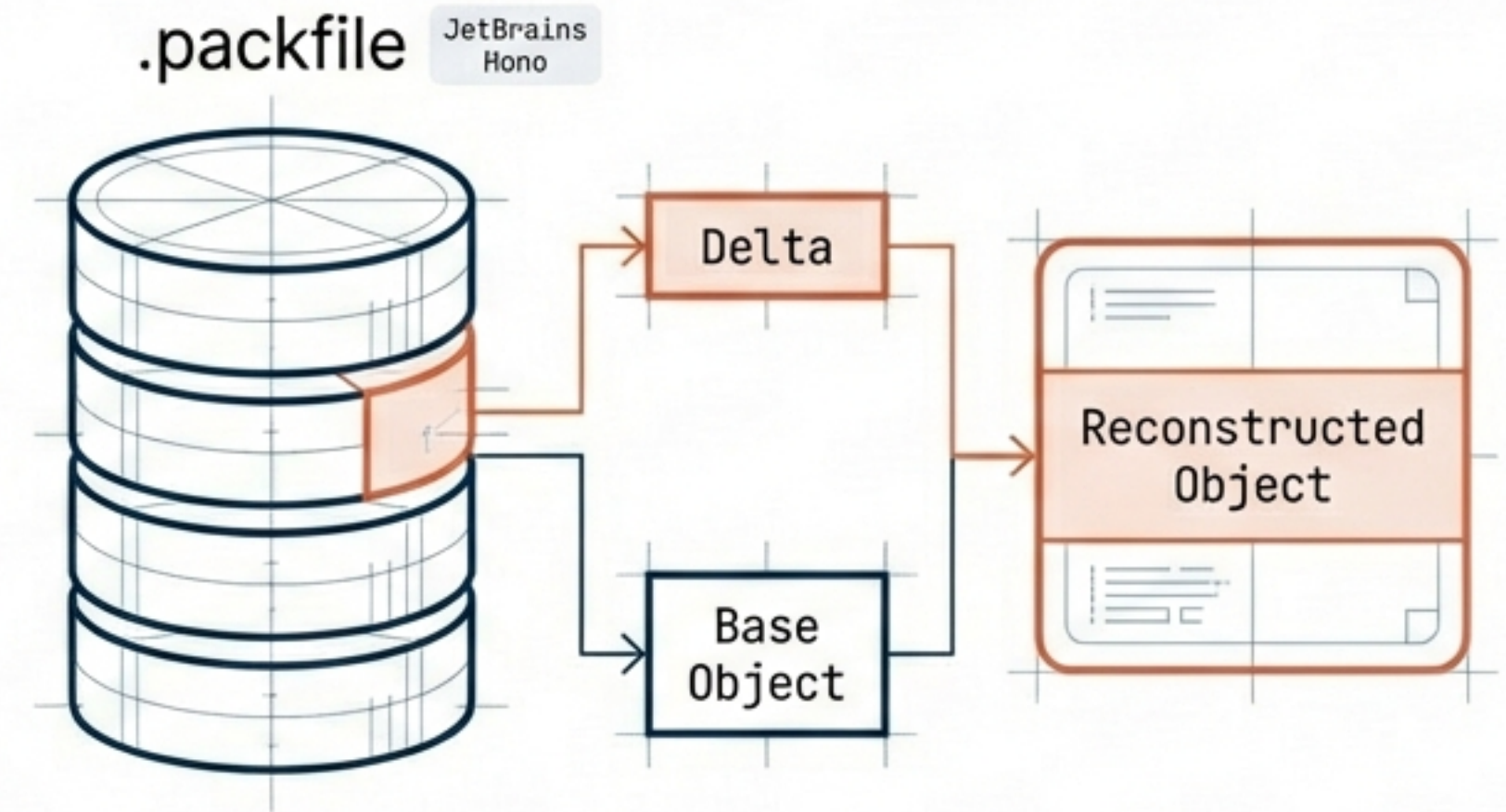
Roadmap: Optimisation & Packfiles

Milestone 4 - Priority P1 (High Complexity)

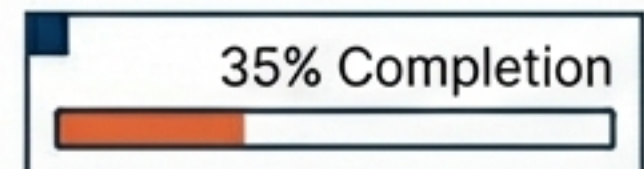
Context: Git compresses objects into .pack files for storage efficiency.

Deliverables List:

- `Git_Object__Reader.read_packed_object()`
- Pack index (v2) parsing
- Delta reconstruction (OFS_DELTA, REF_DELTA)

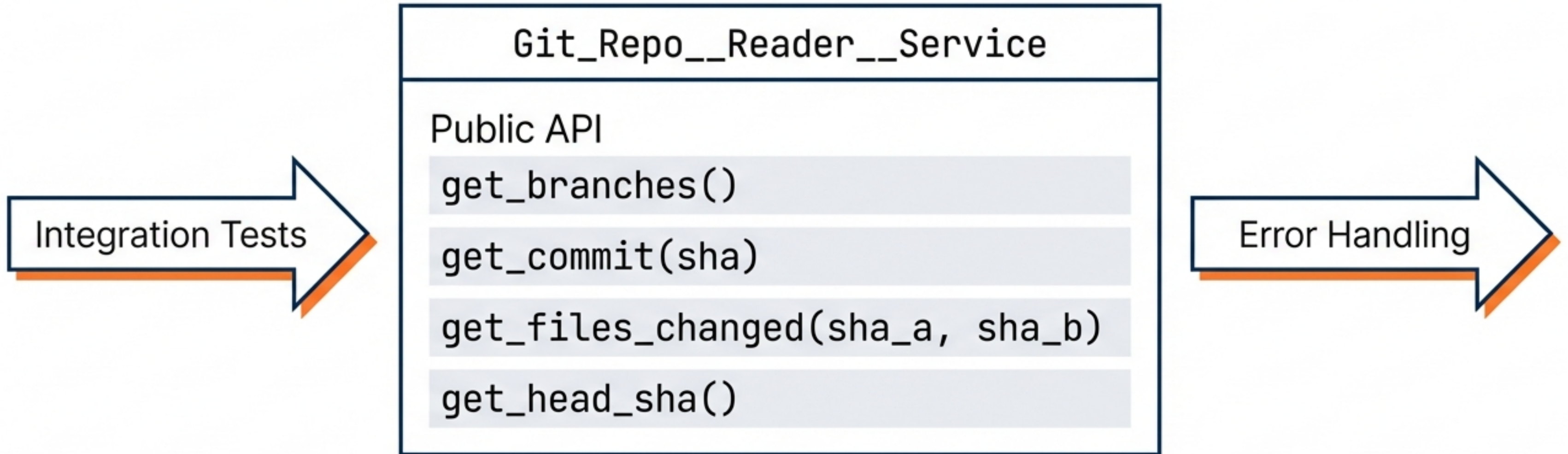


Note: Challenge: Applying delta instructions and calculating offsets is significantly more complex than reading loose files.



Roadmap: The Service API

Milestone 5 - Priority P0



Goal: Provide a clean API where the user calls simple methods and the service handles all zlib and parsing complexity internally.

Success Criteria & Quality Assurance

MVP Checklist

- ✓ Read `HEAD` and resolve to commit `SHA`
- ✓ Parse commit objects fully (including `sha_short`)
- ✓ Return all data as `Type_Safe` schemas
- ✓ Works with zero external dependencies

Quality Standards

- 100% `Type_Safe` compliance: No raw primitives in public API.
- Serialization: Full JSON round-trips.
- Performance: Verified on repositories with 1000+ commits.
- Testing: Comprehensive unit tests with synthetic git data.

Open Questions & Next Steps

1. Packfiles:

Is partial support (returning `None` for packed objects) acceptable for the very first iteration?

2. Error Handling:

Should missing objects return `None` or raise `exceptions`?

3. Legacy Data:

Strategy for handling non-UTF-8 `commit messages` in older repos.

4. Scaling:

Caching strategies for repeated access in large repos.

Status: Ready for Development. | **Reference:** v0.2.26